Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich

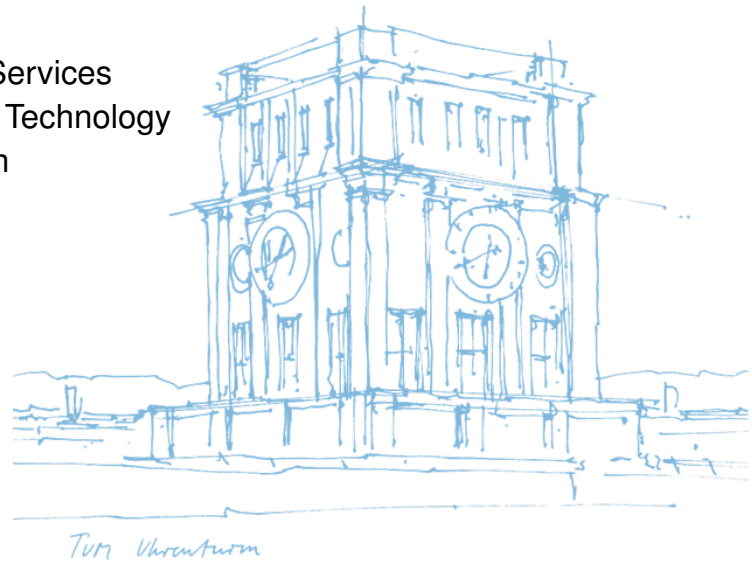
QUIC Improvements Using XDP

Final talk for the Master's Thesis by

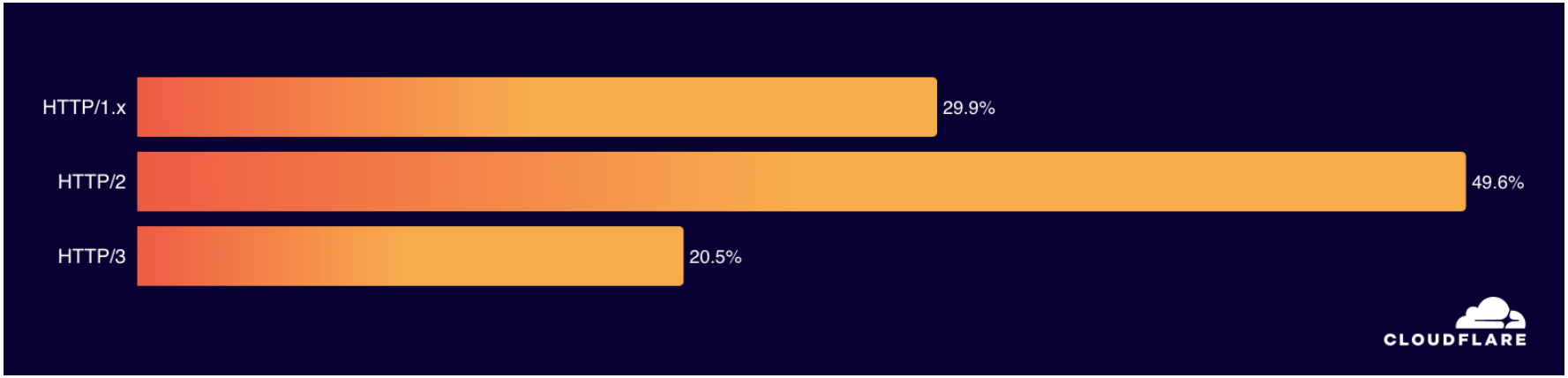
Dominik Ritzenhoff
advised by Johannes Späth, Marcel Kempf

Monday 18th August, 2025

Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich



Motivation



- Roughly 20% of the world's internet traffic now runs on HTTP/3, the protocol built on QUIC[1]
- QUIC throughput is not able to match TCP with increasing bandwidth[2]
- Packet I/O is QUIC's largest bottleneck in terms of its maximum achievable goodput[3]

Major QUIC Implemetnations**

for the implementations that were involved

XDP offers a remedy

Too much of a jump for XDP. Consider centering

* There has been work to approach this bottleneck. Diff datapaths. Mine is XDP

[1] https://radar.cloudflare.com/year-in-review/2024#http-versions

[2] Zhang et al., Quic is not quick enough over fast internet, WWW '24, pp. 2713–2722

[3] Jaeger et al., Quic on the highway, June 2023, pp. 1–9.

Research Questions

- To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?
- Can a taxonomy be created for QUIC optimizations, categorizing them by implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment?

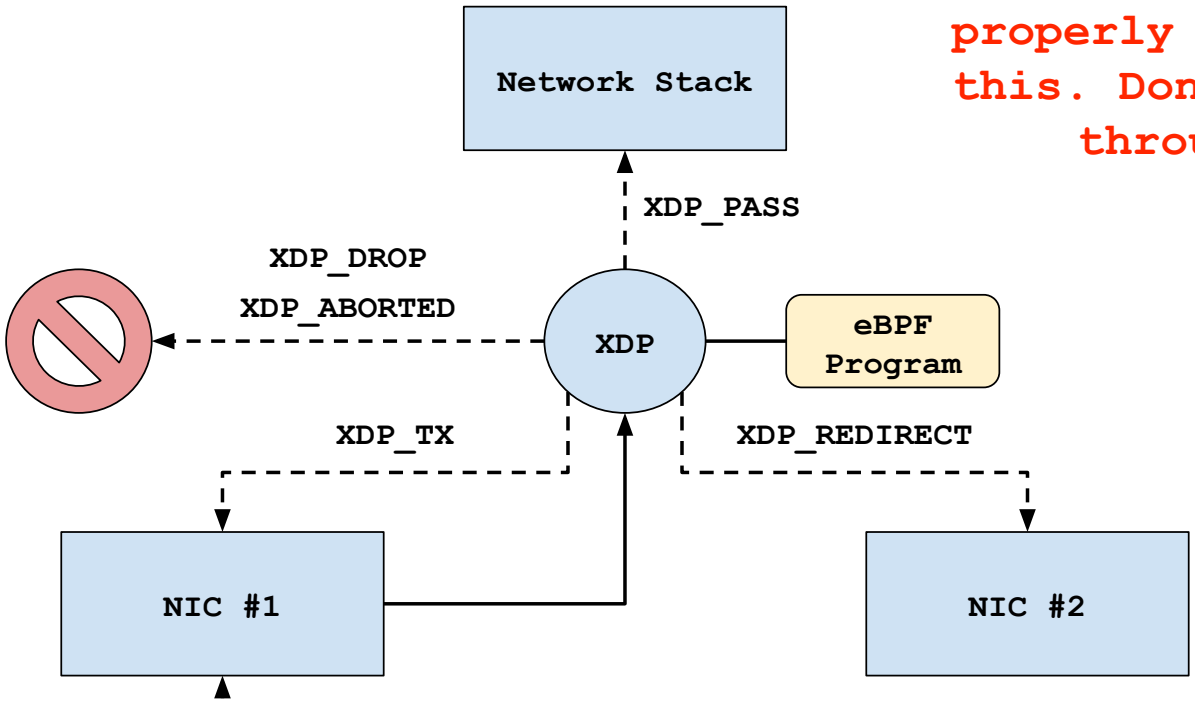
Can we split this up to make it more digestible?

Related work after this and mention alternative datapaths
Only include figure if you have time.

XDP Primer

- XDP is a native Linux hook point for eBPF bytecode to run in
- Decide on an action for each packet: `XDP_DROP`, `XDP_PASS`, `XDP_REDIRECT`, `XDP_ABORTED`, `XDP_TX`

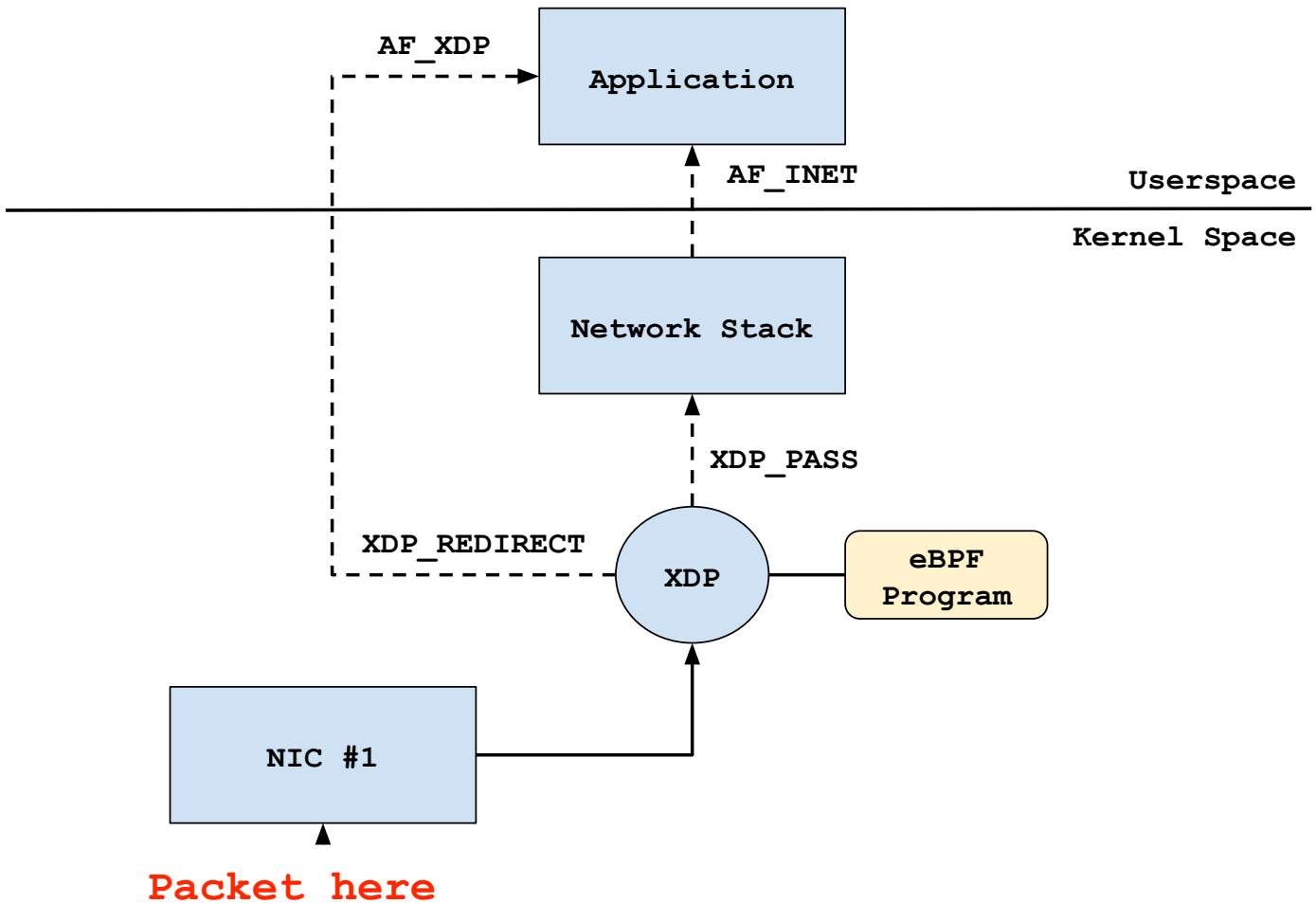
Make sure to properly go over this. Don't rush through



make it clearer that a packet is arriving here

XDP Primer

- AF_XDP is an address family in Linux similar to AF_INET or AF_INET6
- Works with XDP to bypass the network stack
- Helps avoid costly per packet processing

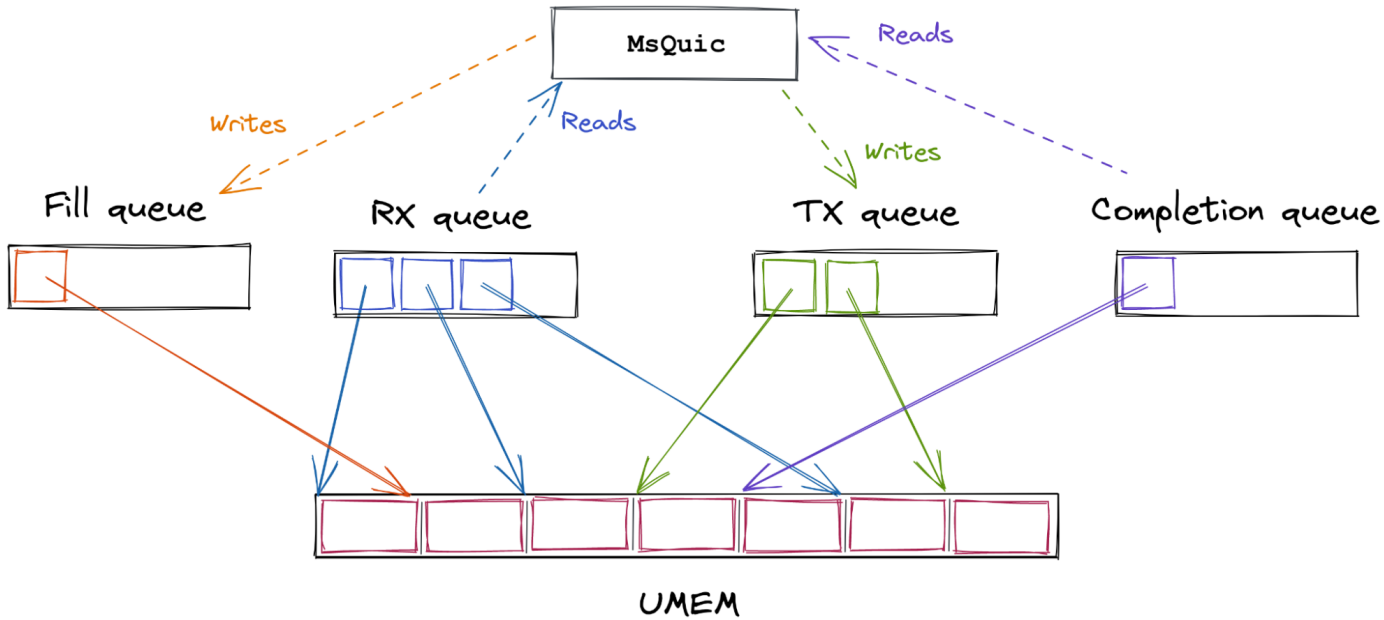


AF_XDP Primer

capitalize

- AF_XDP uses UMEM with tx, rx, completion, and fill rings
- Each ring is single producer, single consumer
- Avoids locking contention
- Need a single system call to trigger multi-packet sends

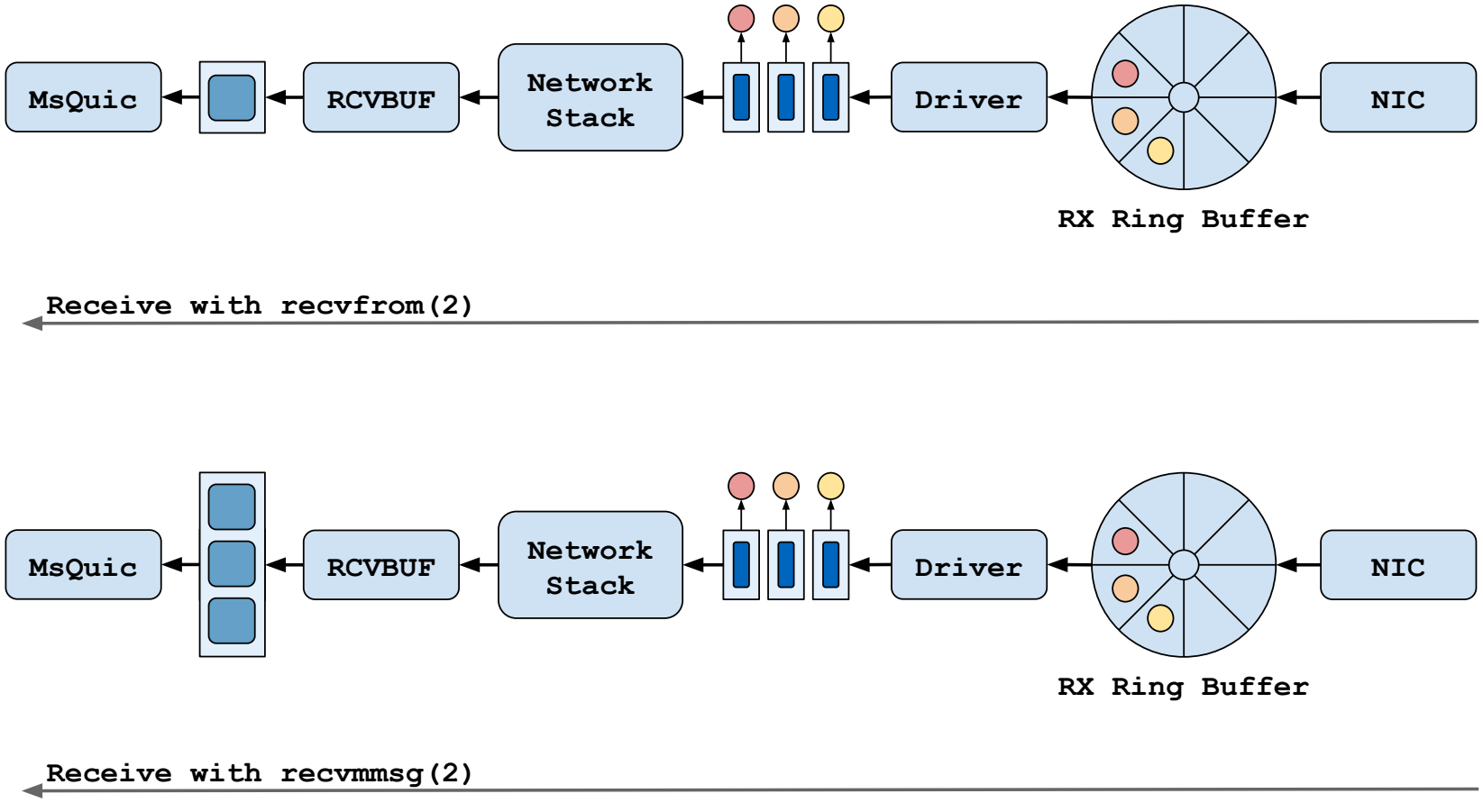
- more general points about what UMEM is and why it's important for AF_XDP



This figure is too complicated. Figure out a diff. way to introduce UMEM

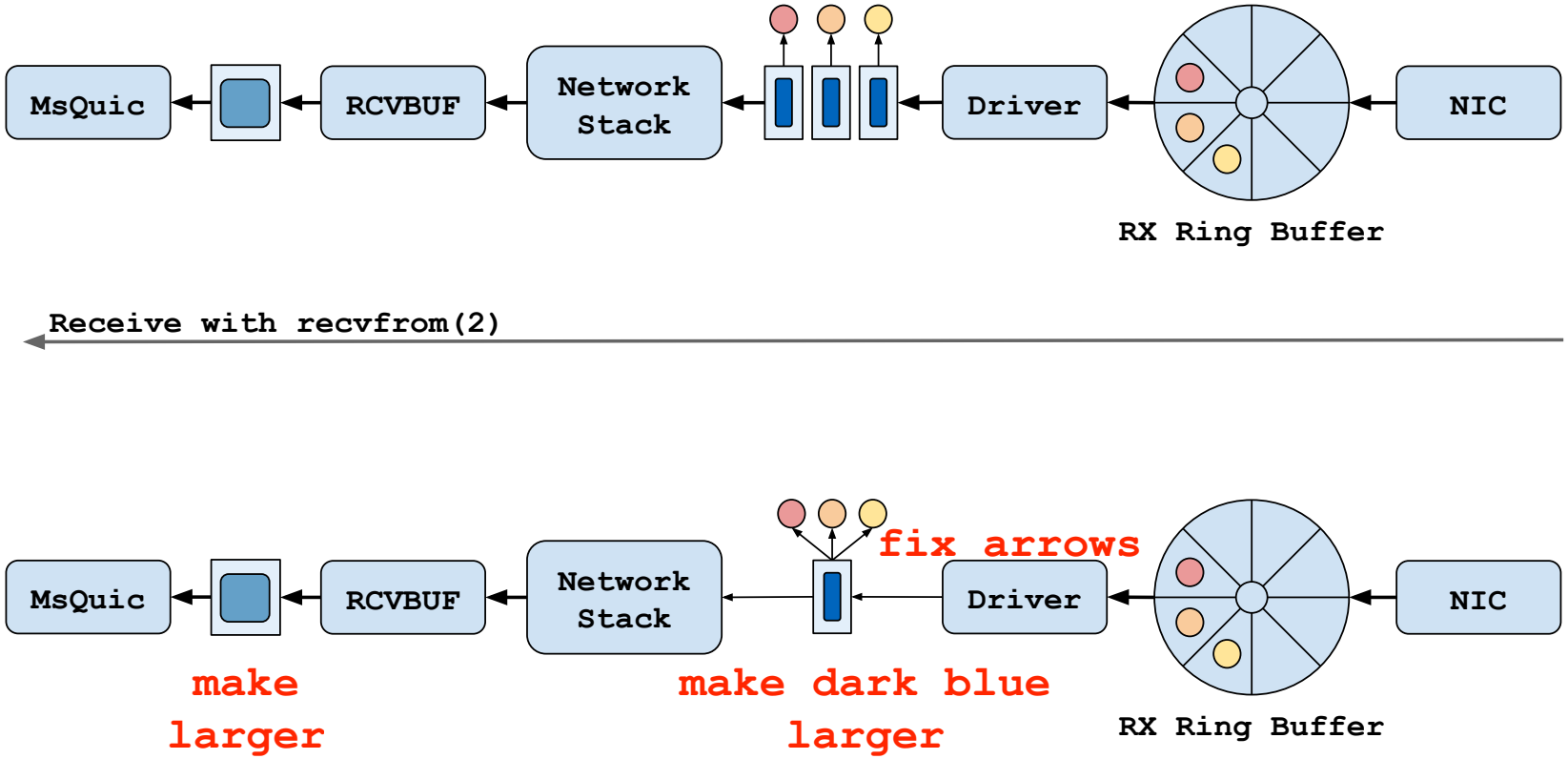
Additional Optimizations

recvmmsg(2)



Additional Optimizations

Generic Receive Offload (GRO)

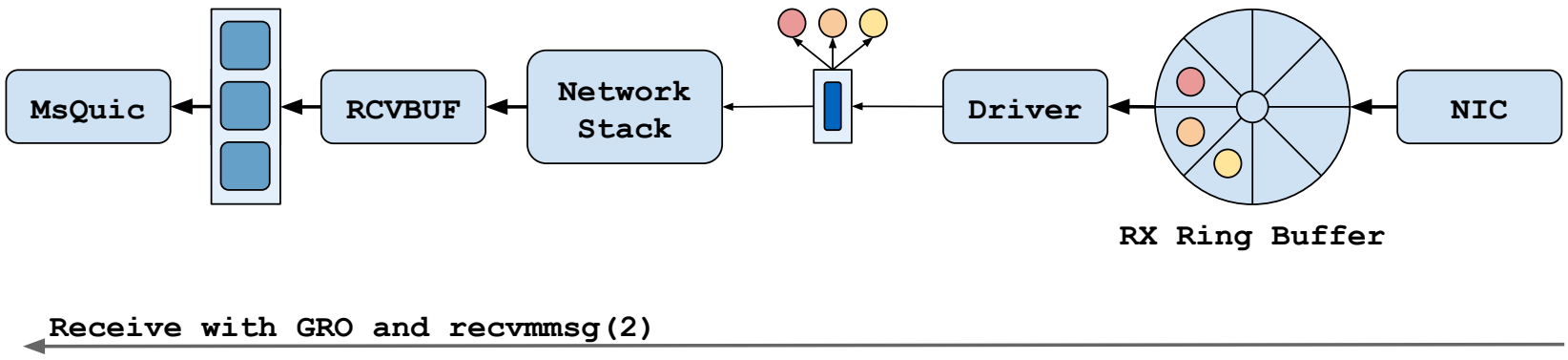


Additional Optimizations

recvmmsg(2) + GRO

drop first two. keep simple instead.
you can just combine two

- recvmmsg(2) system call receives multiple messages from a socket in a single call, which can be more efficient than making multiple recvfrom(2) calls.
- GRO combines network packets into a single larger one, reducing the number of packets the CPU has to process
- A similar optimization exists for sending (i.e. GSO and sendmmsg(2))



show figure. Pause. Then talk about sending-side equivalent

Approach

Throughput

Goodput

- Patch MsQuic[4] to support AF_XDP
- Run throughput tests in a client-server topology
- 20 iterations with 8GB file size using Dogecoin[5] and Opulous[6] testbeds.



- weak setup and strong setup. put specs on main part of slide. Just important stuff
- explain why do I have these two setups.

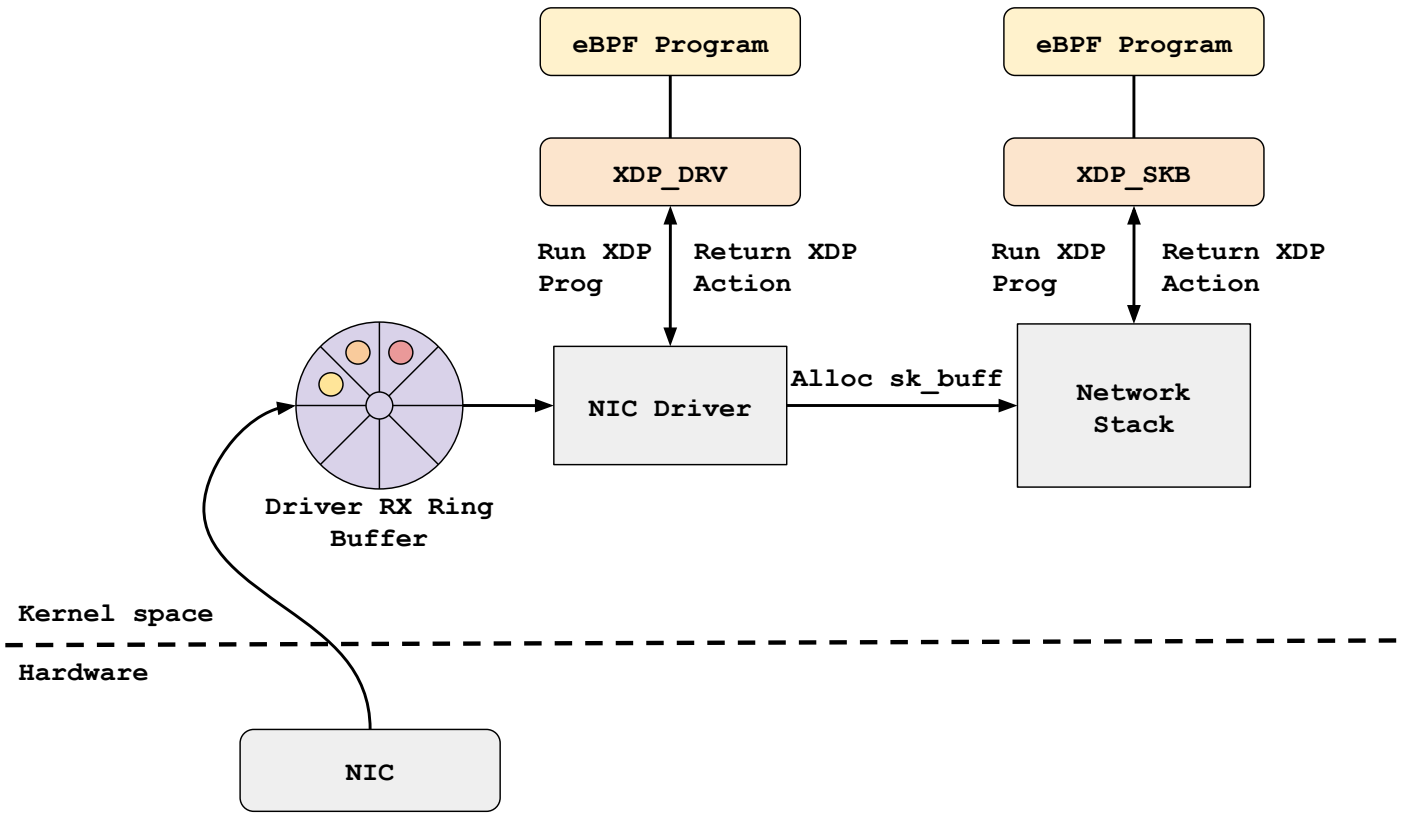
[4] https://github.com/microsoft/msquic

[5] Dogecoin: Intel Xeon D-1518 @ 2.20GHz 4-core CPU, 32 GB of RAM, and Intel X552 10 GbE SFP+ NICs, using ixgbe driver (version 6.1.0-17-amd64) over a 10 Gbase link.

[6] Opulous: AMD EPYC 9354 @ 3.25 GHz 32-core CPU, 768 GB of RAM, and Mellanox MT28908 ConnectX-6 NICs, using mlx5_core driver (version 6.1.0-17-amd64) over a 100 Gbase link.

XDP Hook Points

- Two different XDP hook points: `XDP_DRV` and `XDP_SKB`
- `XDP_SKB` requires additional processing over `XDP_DRV`

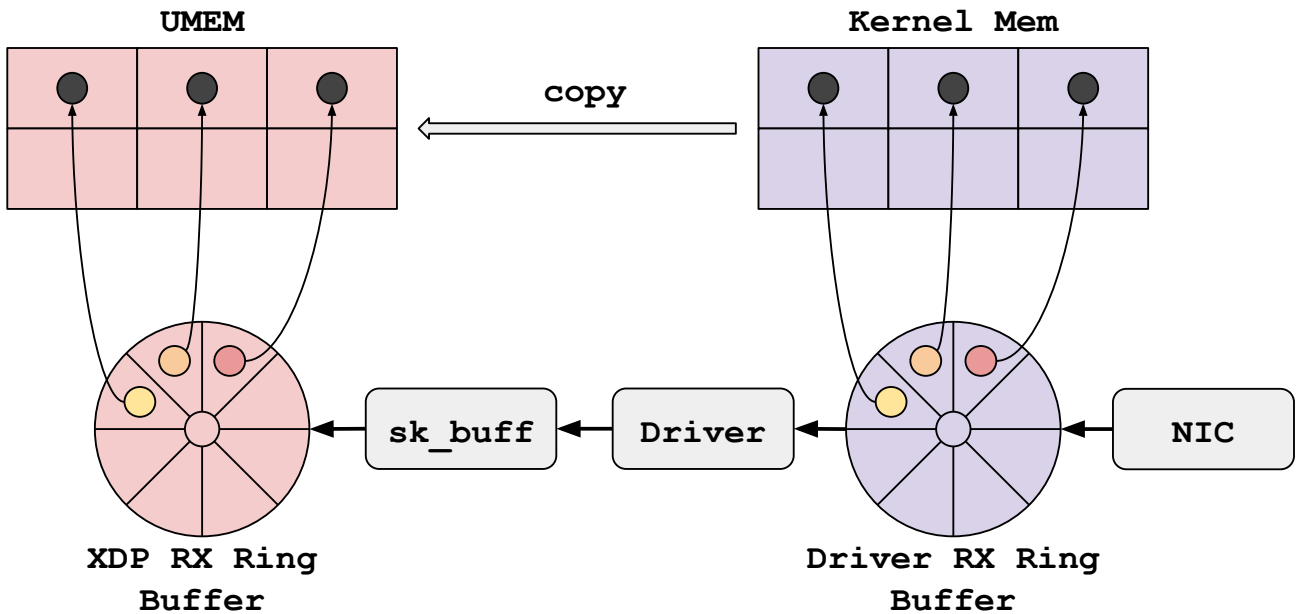


XDP Types

XDP_SKB

- Problem w/ two modes. Introduce zc
- Describe benefit

- Packet is copied by NIC into kernel memory and an sk_buff is allocated
- Packet is copied from kernel memory into the UMEM region

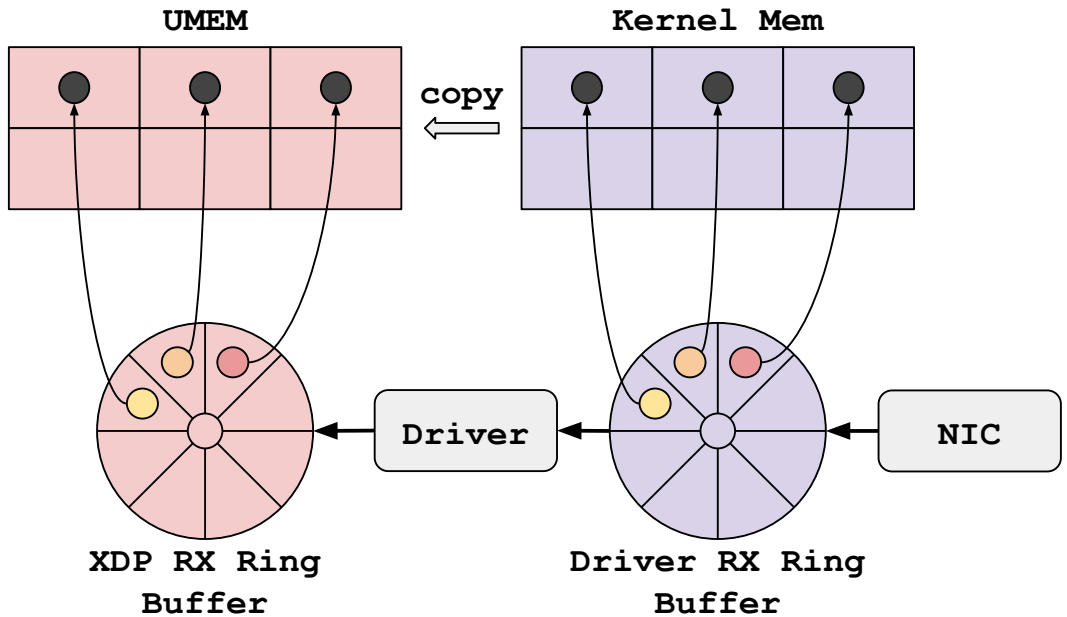


Move these into Appendix for further visuals if necessary

XDP Types

XDP_DRV

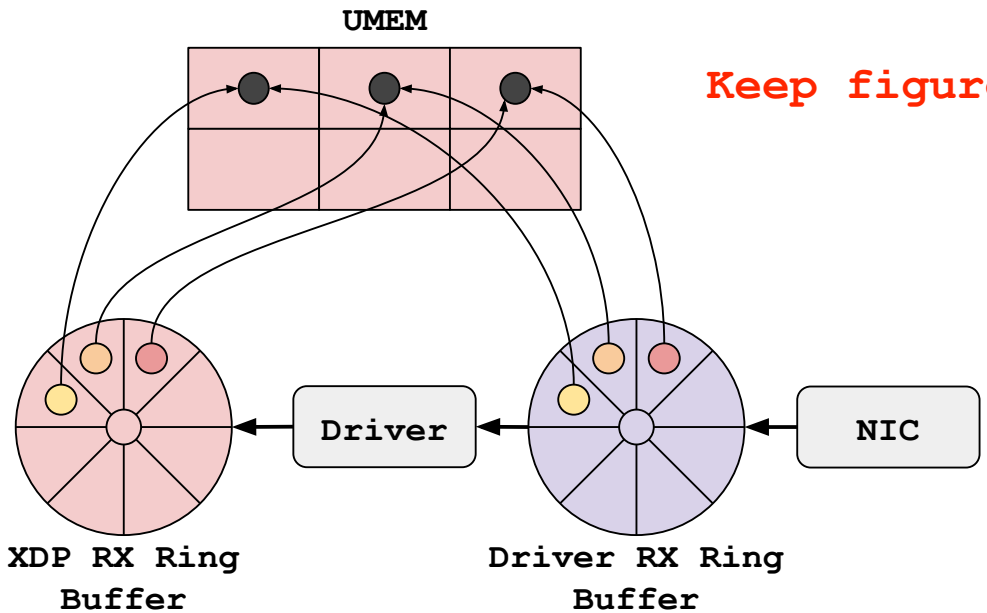
- Packet is copied by NIC into kernel memory; no sk_buff this time
- Packet is copied from kernel memory into the UMEM region



XDP Types

XDP_DRV with Zero-Copy (zc)

- Packet is copied by NIC directly into UMEM region; an additional copy is therefore completely avoided
- A similar process exists for sending

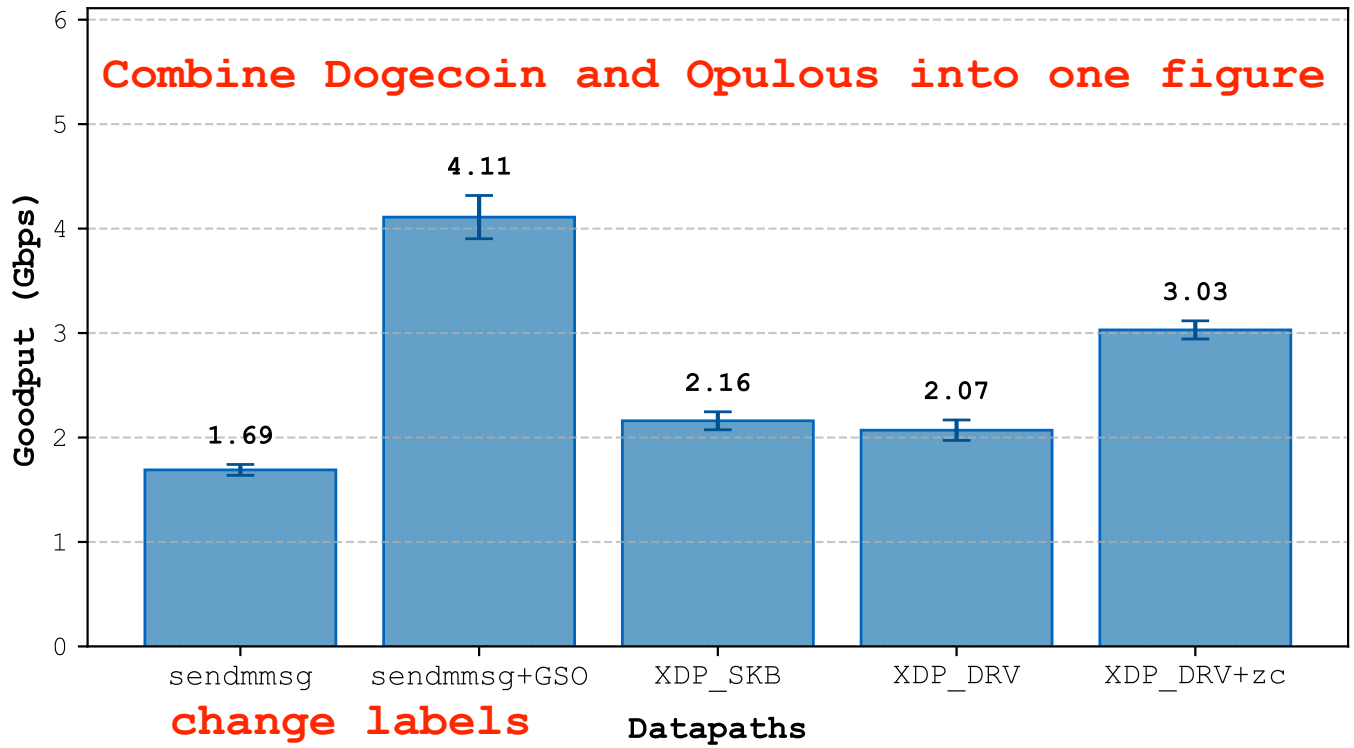


Measurements

AF_XDP on the Server (Dogecoin)

Measurement node -> server
Testbed -> the whole thing
Dogecoin is not a testbed

- `XDP_DRV+zc` exhibits higher goodput than substitutes but remains lower than `sendmmsg+GSO`
- `GSO` makes the difference within the default datapath

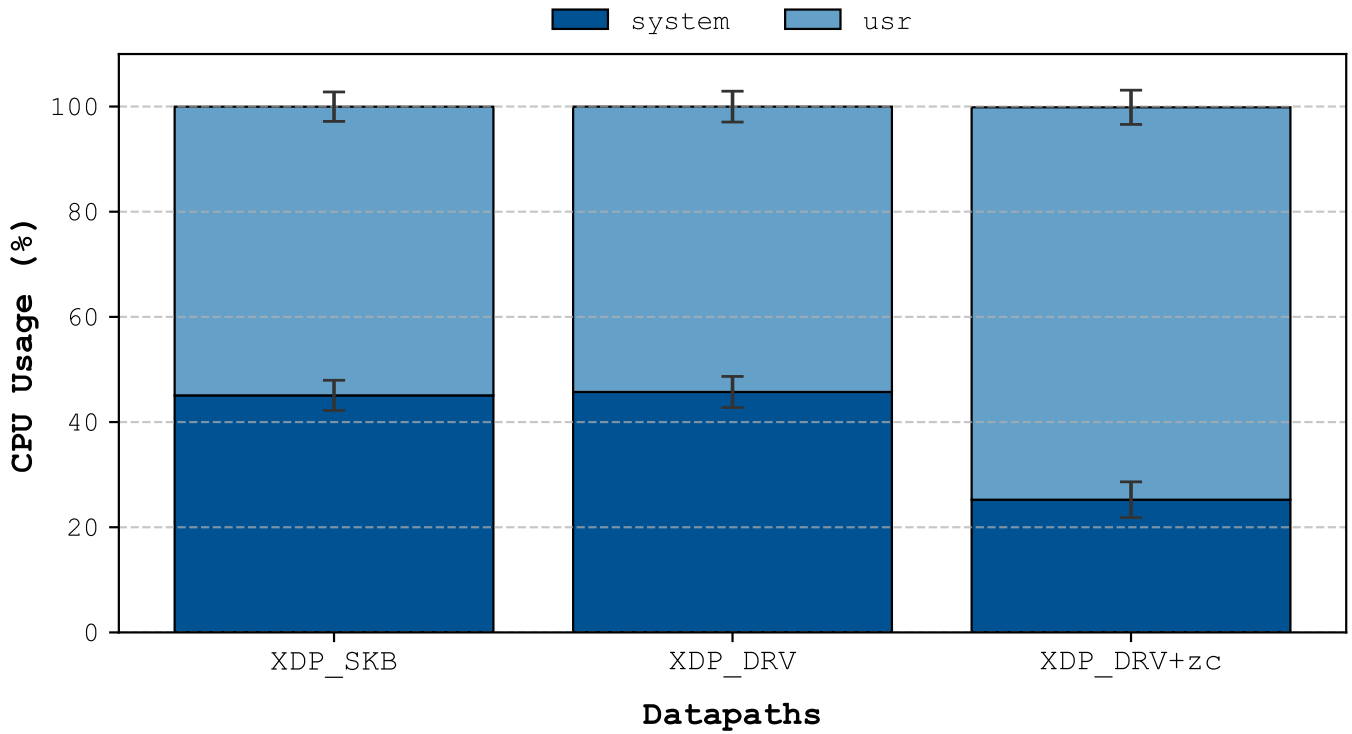


play around w/ combinations send+rcv or dogecoin+opulous

Measurements

AF_XDP on the Server, continued

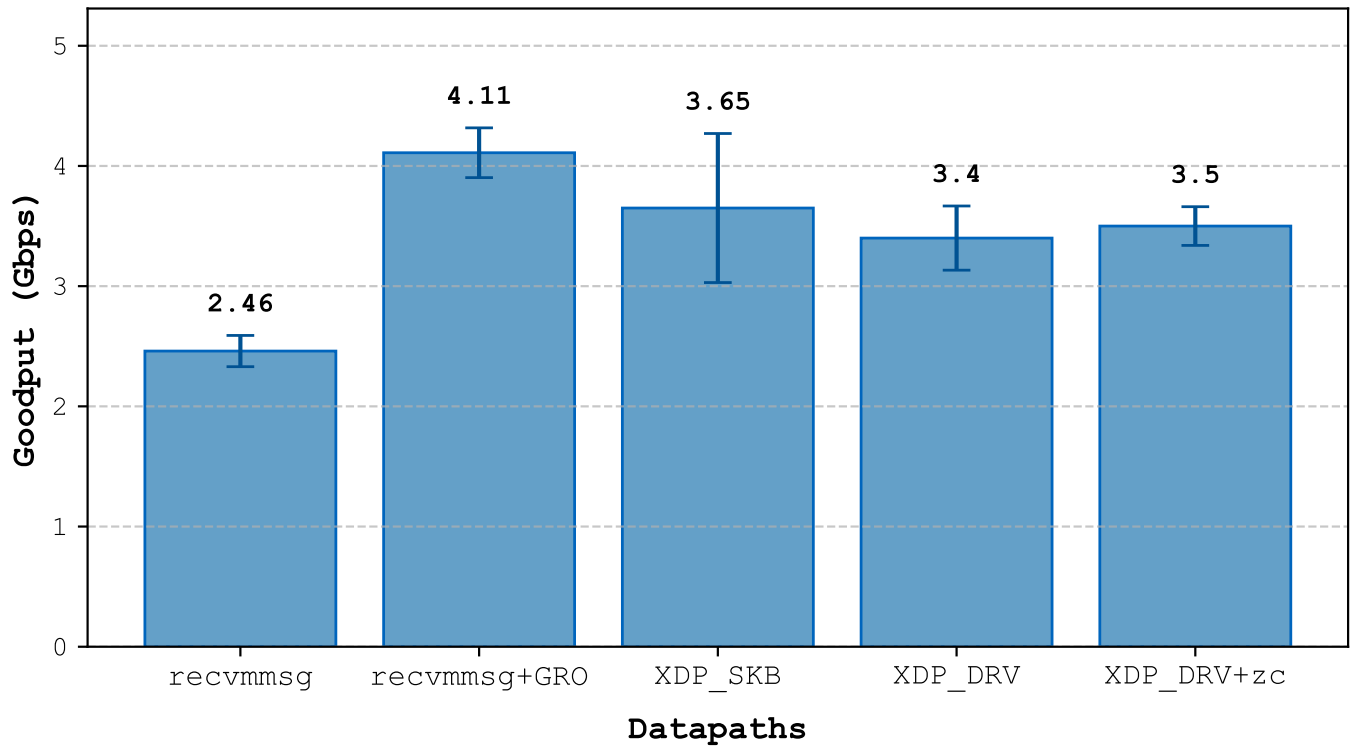
- System CPU usage for `XDP_SKB` and `XDP_DRV` higher than `XDP_DRV+zc`
- We used a perf trace to pinpoint the increased time spent copying data from the UMEM to kernel memory



Measurements

AF_XDP on the Client (Dogecoin)

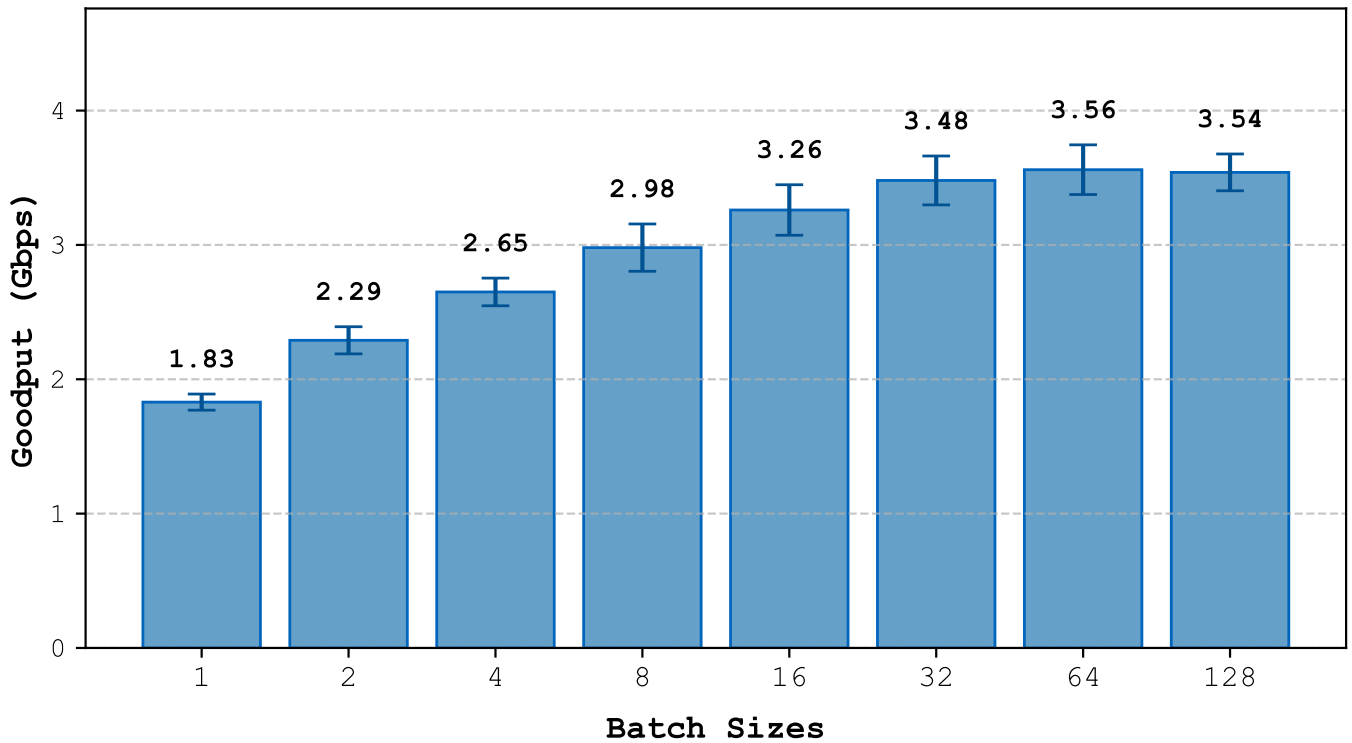
- Surprisingly, the XDP alternatives are no longer statistically significant from each other
- Talk about large `XDP_SKB` std dev. here?



Measurements

RX Batch Sizes with AF_XDP on the Client (Dogecoin)

- Goodput increases as RX batch size increases until 32
- The bottleneck changes; MsQuic must no longer wait for packets and instead spends its time on processing

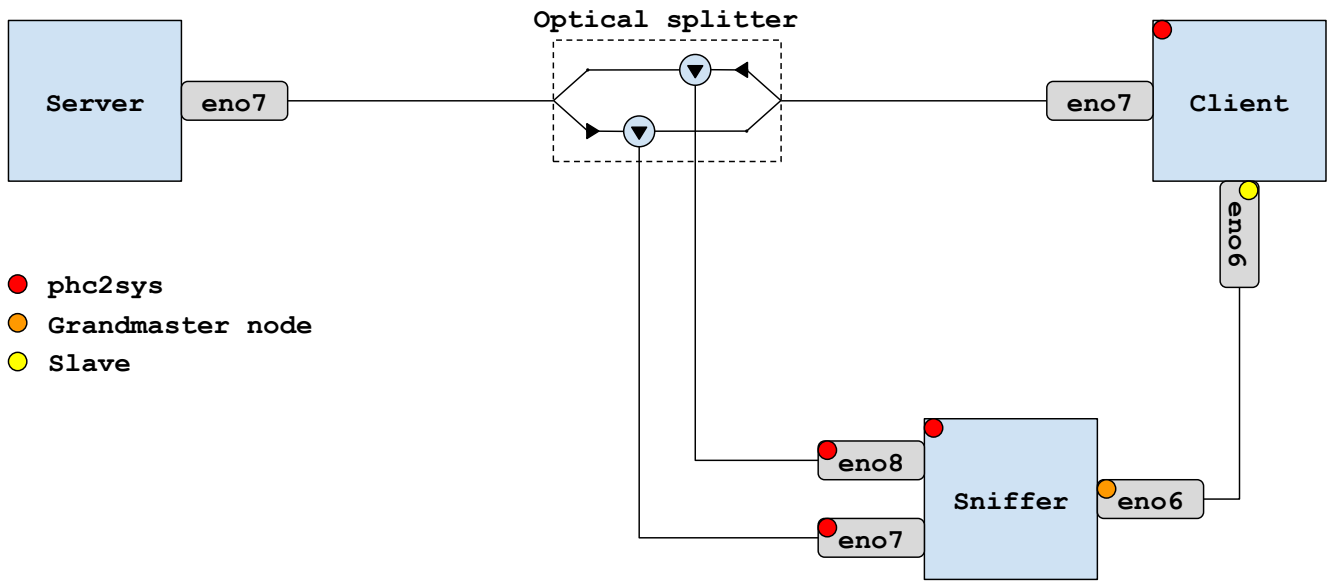


Consider moving to appendix if lack of time

Approach

NIC-to-Userspace Packet Traversal Time

- Sync NICs and system times using Precision Time Protocol (PTP)[7]
- 1 iteration with 128MB file size using Dogecoin[8]
- Generates more than 100k NIC-to-Userspace packet time differences



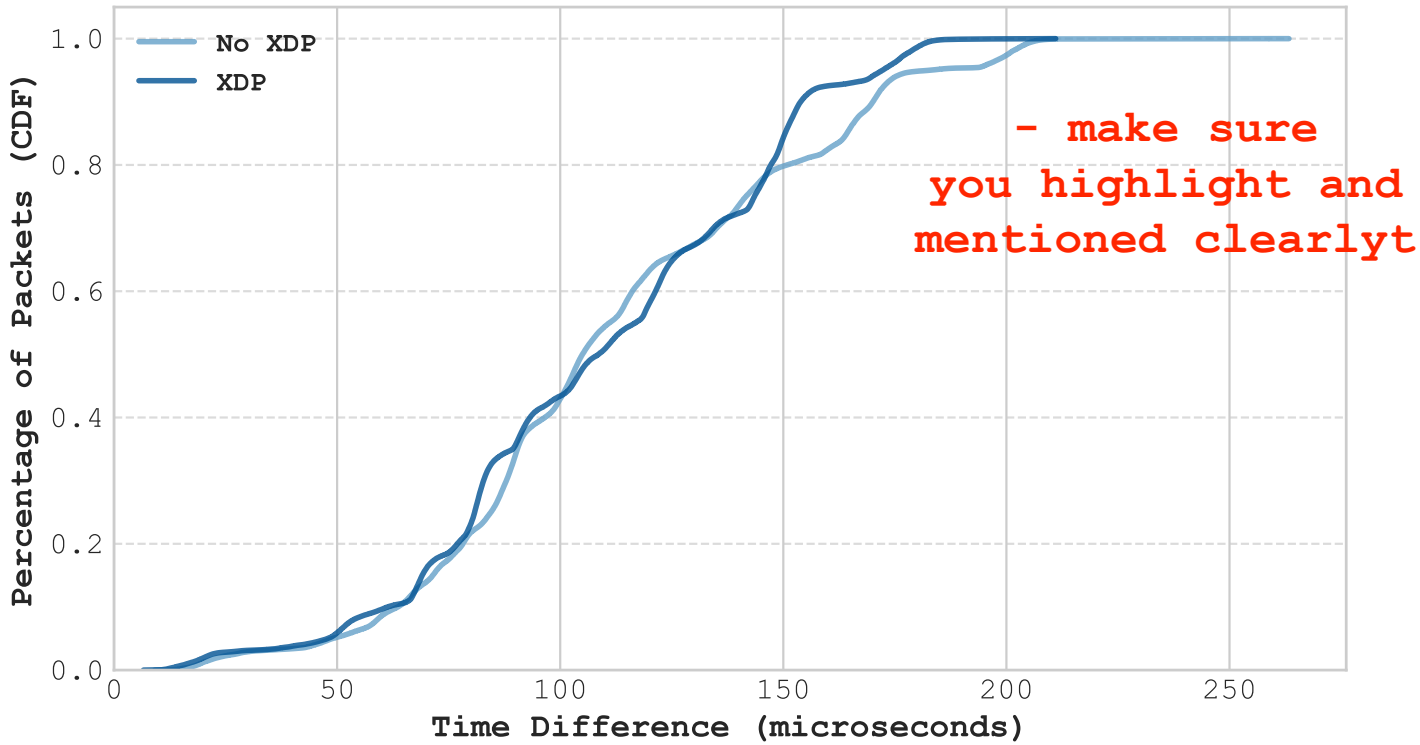
- Mention it took a lot of time to get working
- Get rid of colors and just say shit was synced

[7] https://github.com/richardcochran/linuxptp

[8] Dogecoin: Intel Xeon D-1518 @ 2.20GHz 4-core CPU, 32 GB of RAM, and Intel X552 10 GbE SFP+ NICs, using ixgbe driver (version 6.1.0-17-amd64) over a 10 Gbase link.

Measurements

- XDP and default datapath NIC-to-userspace times mostly remain in parity
- The long tail in the non-XDP case is likely due to additional queues in the receive path



Related Work

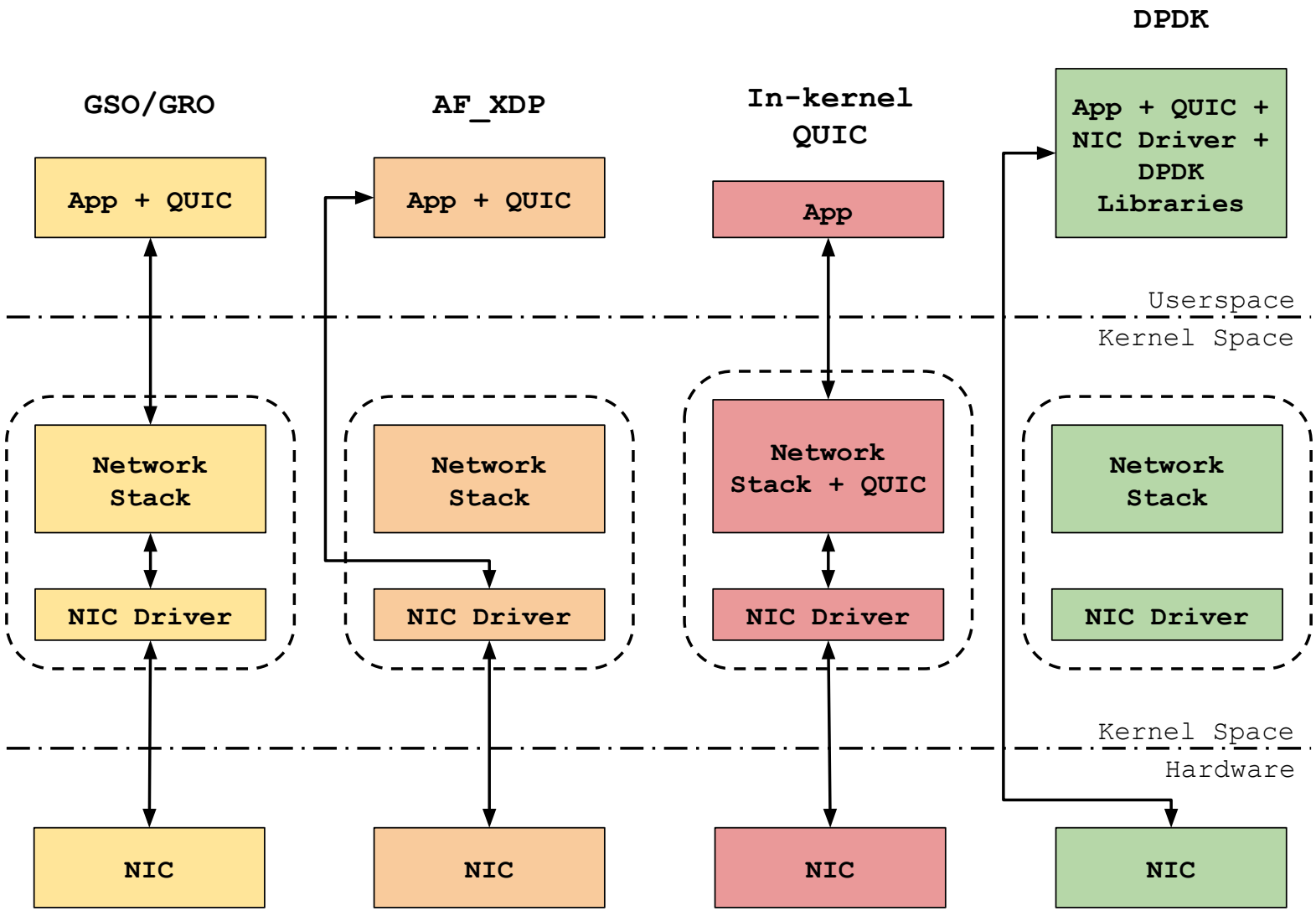
- DPDK[9] enables kernel bypass and allows application to connect directly to the NIC, reducing kernel overhead
- lxin-quic[10] brings QUIC into the kernel, reducing context switches and copying between kernel and userspace
- *QUIC Performance Improvements Using Socket Optimizations and Kernel Bypass*[11] investigates methods to improve QUIC performance by means of patching MsQuic with DPDK and using socket optimizations

[9] https://www.dpdk.org/

[10] https://github.com/lxin/quic

[11] Späth, J. QUIC performance improvements using socket optimizations and kernel bypass. Master's thesis, TU Munich, 2023.

QUIC Datapaths



Taxonomy of QUIC Datapaths

Consider simplified version of this table.
Select dimensions of this table.

Metric	GSO/GRO	AF_XDP	In-kernel QUIC	DPDK
Impl. Complexity	Low	High	High	High
Standardized APIs	✓	×	✓	×
Kernel Integration	High	Medium	High	Low
NIC Sharing	✓	✓	✓	×
Hardware Agnostic	High	High*	High	Low
Maturity	High	Medium	Low	High
Tooling	High	High	High	Medium
Privileged Access	✓	✓	✓	✓

Can mention there is more to this. More can be found in thesis.

Conclusion

consider top-level answer for each RQ

- To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?
  - AF_XDP is not able to achieve a higher goodput over recvmmsg()+GRO
  - AF_XDP NIC-to-userspace times remain consistent with the default datapath
  - However, optimization potential exists

- Can a taxonomy be created for QUIC optimizations, categorizing them by implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment?
  - Yes, each datapath offers its unique set of tradeoffs

see if you can add the simplified table here. Hopefully you talked about every part of the table already

Next Steps

Future Work

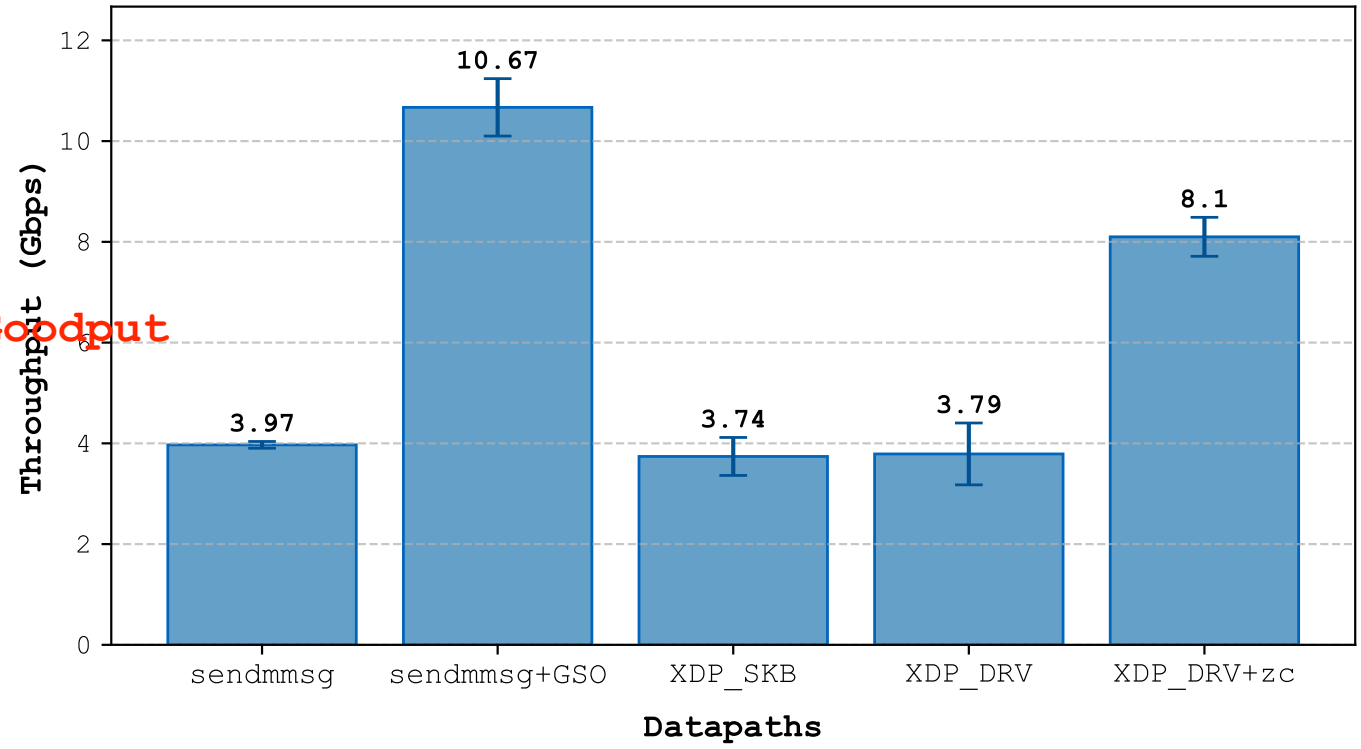
- Sometimes future work and conclusion are on one slide for discussion.
- Play around w/ this.

- Enable GRO for AF_XDP, for which driver support has been increasing [9]
- Increase the TX ring batch size from its default of 1, which may offer similar speedups as seen in the RX batch size case
- Perform throughput and latency measurements across all QUIC datapaths on the same testbed

Appendix

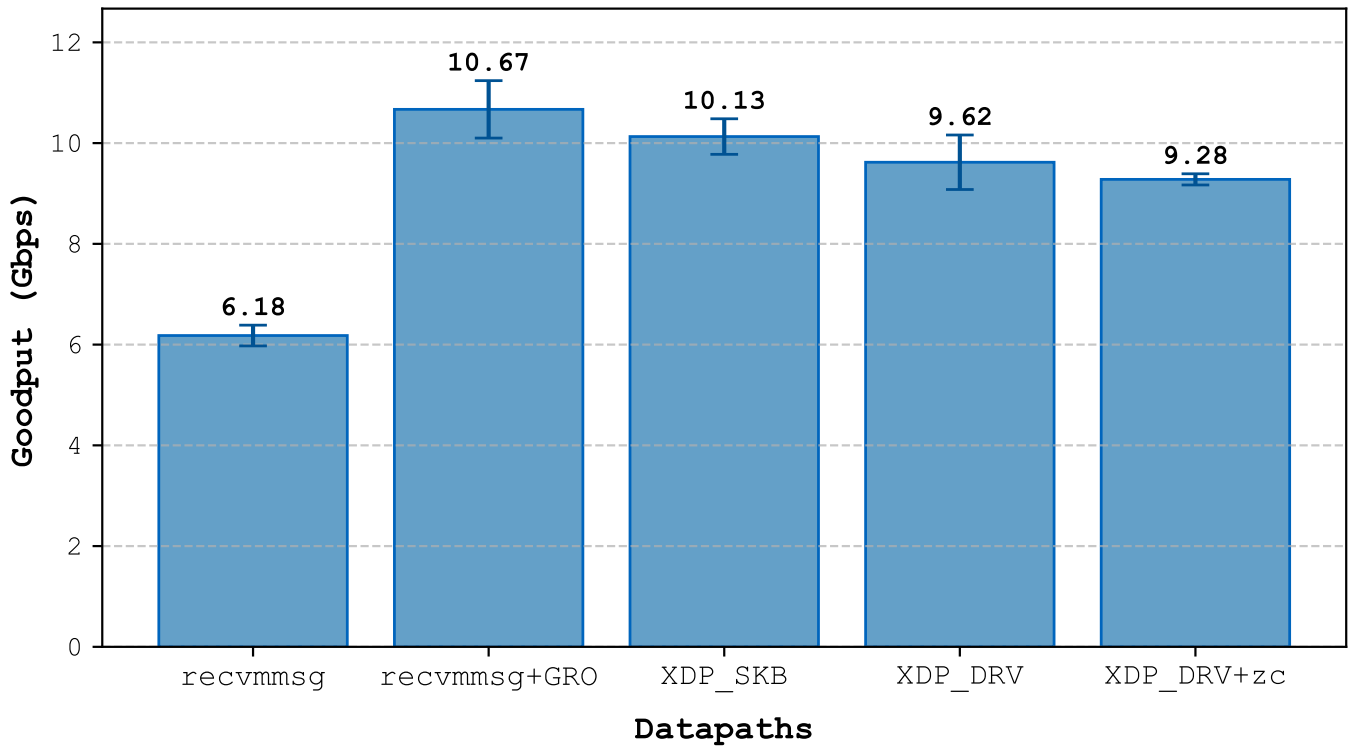
AF_XDP on the Server (Opulous)

Change to Goodput



Appendix

AF_XDP on the Client (Opulous)



Bibliography

[1] Cloudflare.
Cloudflare radar year in review 2024, 2024.
Accessed: 2024-12-14.

[2] Xumiao Zhang, Shuowei Jin, Yi He, Ahmad Hassan, Z. Morley Mao, Feng Qian, and Zhi-Li Zhang.
Quic is not quick enough over fast internet.
In *Proceedings of the ACM Web Conference 2024*, WWW '24, page 2713–2722. ACM, May 2024.

[3] Benedikt Jaeger, Johannes Zirngibl, Marcel Kempf, Kevin Ploch, and Georg Carle.
Quic on the highway: Evaluating performance on high-rate links.
pages 1–9, 06 2023.

[4] Marten Gartner.
Recapitulating af_xdp.
https://medium.com/high-performance-network-programming/recapitulating-af-xdp-ef6c1ebead8, 2022.
Figure: "The flow of packets using a normal socket and AF_XDP", retrieved from the article.

[5] Bastien Dhiver.
A story about af_xdp, network namespaces, and a cookie.
https://blog.cloudflare.com/a-story-about-af-xdp-network-namespaces-and-a-cookie/, 2022.
Cloudflare Blog.

[6] Linux Foundation.
Data plane development kit (DPDK), 2015.

[7] Xin Long.
In-kernel quic implementation with userspace handshake.
https://github.com/lxin/quic, 2023.
Accessed: 2025-04-20.

Bibliography

[8] Johannes Späth.
Quic performance improvements using socket optimizations and kernel bypass.
Master's thesis, Technical University of Munich, Garching, Bavaria, 2023.

[9] Lorenzo Bianconi, Eelco Chaudron, Jesper Dangaard Brouer, and Toke Høiland-Jørgensen.
Bringing tso/gro and jumbo frames to xdp.
Linux Plumbers Conference, 2021.
Contribution to Linux Plumbers Conference 2021.